Chapitre 0 — Avant tout | Partie D — Des formes | leçon 0.51 sur 153 | difficulté 1 / 10

0.51. Jusqu’au plus grand : une boucle

Une boucle for fait grandir la taille de 1 à 8 : huit carrés emboîtés, jusqu’au plus grand que l’écran tienne.

1. Ce qu’il faut comprendre

Le 0.50 répétait la même ligne, avec une taille qui augmente de 1 à chaque fois. Quand seul un nombre change, et toujours de la même façon, on écrit la ligne **une seule fois**, dans une **boucle**.

Ce qui est nouveau ici : la taille est une **variable**, `taille`, que la boucle `for` fait passer par 1, 2, 3… jusqu’à **8**. À chaque tour de boucle, `carre` reçoit la taille du moment.

`taille <= 8` veut dire « tant que la taille est plus petite **ou égale** à 8 » : la boucle fait donc aussi le tour de taille 8, puis s’arrête.

Pourquoi 8 : c’est le plus grand carré que l’écran tienne. Autour de (10, 8), il va de la colonne 2 à 18 et de la ligne 0 à 16. Le centre a été choisi pour ça (voir le 0.49).

La vitesse, 50 ms par pas, est plus rapide : huit carrés à 250 ms dureraient plus d’une minute. Elle s’écrit toujours en clair ; la taille, elle, peut être une variable.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <carre>` → 0.33.3. La fonction carre() — tourner en carré
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **for** — Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <carre>  // une tuile qui tourne en carré`	Demande à la console « carre » (ce que c'est : une tuile qui tourne en carré). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en carré ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)

`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// La boucle, morceau par morceau :`	Un commentaire : une phrase pour celui qui lit (« La boucle, morceau par morceau : »). La console l'ignore.
`//   for (uint8_t taille = 1; taille <= 8; taille++)`	Un commentaire : une phrase pour celui qui lit (« for (uint8_t taille = 1; taille <= 8; taille++) »). La console l'ignore.
`//        |                  |           |`	Un commentaire : une phrase pour celui qui lit (« ||| »). La console l'ignore.
`//        |                  |           +--` après chaque tour : taille + 1	Un commentaire : une phrase pour celui qui lit (« || +-- après chaque tour : taille + 1 »). La console l'ignore.
`//        |                  +---------------` tant que taille ≤ 8 (8 compris)	Un commentaire : une phrase pour celui qui lit (« | +---------- tant que taille ≤ 8 (8 compris) »). La console l'ignore.
`//        +---------------------------------- on commence à la taille 1`	Un commentaire : une phrase pour celui qui lit (« +---------- on commence à la taille 1 »). La console l'ignore.
`//   tour 1 : carre(10, 8, ALPHABET[0], 1, 1, 50, 1);   3 × 3`	Un commentaire : une phrase pour celui qui lit (« tour 1 : carre(10, 8, ALPHABET[0], 1, 1, 50, 1); 3 × 3 »). La console l'ignore.
`//   tour 2 : carre(10, 8, ALPHABET[0], 2, 1, 50, 1);   5 × 5`	Un commentaire : une phrase pour celui qui lit (« tour 2 : carre(10, 8, ALPHABET[0], 2, 1, 50, 1); 5 × 5 »). La console l'ignore.
`//   …`	Un commentaire : une phrase pour celui qui lit (« … »). La console l'ignore.
`//   tour 8 : carre(10, 8, ALPHABET[0], 8, 1, 50, 1);   17 × 17, le plus grand`	Un commentaire : une phrase pour celui qui lit (« tour 8 : carre(10, 8, ALPHABET[0], 8, 1, 50, 1); 17 × 17, le plus grand »). La console l'ignore.
`// Le plus grand, autour du centre (10, 8) :`	Un commentaire : une phrase pour celui qui lit (« Le plus grand, autour du centre (10, 8) : »). La console l'ignore.
`//   colonnes 2 à 18, lignes 0 à 16 : il tient entier dans l'écran`	Un commentaire : une phrase pour celui qui lit (« colonnes 2 à 18, lignes 0 à 16 : il tient entier dans l'écran »). La console l'ignore.
`//   (20 colonnes de 0 à 19, 18 lignes de 0 à 17).`	Un commentaire : une phrase pour celui qui lit (« (20 colonnes de 0 à 19, 18 lignes de 0 à 17). »). La console l'ignore.
`for (uint8_t taille = 1; taille <= 8; taille++) {`	Une boucle qui compte : taille part de 1 ; on continue tant que taille est plus petit ou égal à 8 ; et après chaque tour, taille augmente de 1.
`carre(10, 8, ALPHABET[0], taille, 1, 50, 1);   // 50 ms par pas : plus vite`	Appel de la fonction carre de la console, avec les arguments 10, 8, ALPHABET[0], taille, 1, 50, 1. Ce qu'elle fait : un carré parfait AUTOUR de (x, y) : taille 1 fait 3 × 3, taille 2 fait 5 × 5 (2 × taille + 1). sens 1 : aiguilles d'une montre, -1 : l'autre sens. vitesse : millisecondes par pas, écrite en clair (250). La lettre part du centre, fait ses tours et y revient. Ou, les sept réglages sous un nom : Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 3 }; carre(ronde);. (Le commentaire le dit : « 50 ms par pas : plus vite ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();          // les huit carrés sont faits : le A est au centre`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « les huit carrés sont faits : le A est au centre ».)

`}`	L’accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L’accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l’œil le lit. Il fait **1368 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l’allumage  — la mise en route de la console
  carre()         440 o  ← main()  — une fonction fournie par le langage
  main()           57 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l’image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l’écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l’adresse d’une case de l’écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l’écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 4    52 o sur place, dans carre()  + AdresseCase (10 o)
  effacer() × 3    45 o sur place, dans carre()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 4    12 o sur place, dans carre(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  carre()   × 1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

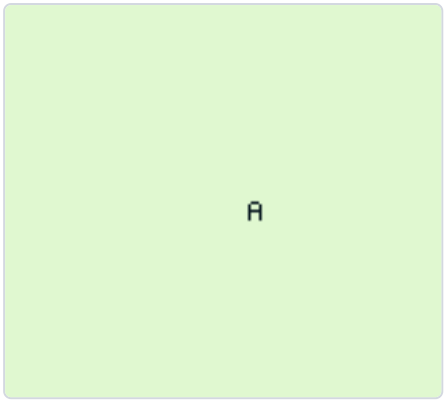
```
int main() {
  // La boucle, morceau par morceau :
  //
  //   for (uint8_t taille = 1; taille <= 8; taille++)
  //        |                   |            |
  //        |                   |            +-- après chaque tour : taille + 1
  //        |                   +--------------- tant que taille ≤ 8 (8 compris)
  //        +------------------------------------ on commence à la taille 1
  //
  //   tour 1 : carre(10, 8, ALPHABET[0], 1, 1, 50, 1);    3 × 3
  //   tour 2 : carre(10, 8, ALPHABET[0], 2, 1, 50, 1);    5 × 5
  //   …
  //   tour 8 : carre(10, 8, ALPHABET[0], 8, 1, 50, 1);    17 × 17, le plus grand
  //
  // Le plus grand, autour du centre (10, 8) :
  //   colonnes 2 à 18, lignes 0 à 16 : il tient entier dans l'écran
  //   (20 colonnes de 0 à 19, 18 lignes de 0 à 17).
  for (uint8_t taille = 1; taille <= 8; taille++) {
    carre(10, 8, ALPHABET[0], taille, 1, 50, 1);   // 50 ms par pas : plus vite
  }

  while (true) {
    image();          // les huit carrés sont faits : le A est au centre
  }
}
```

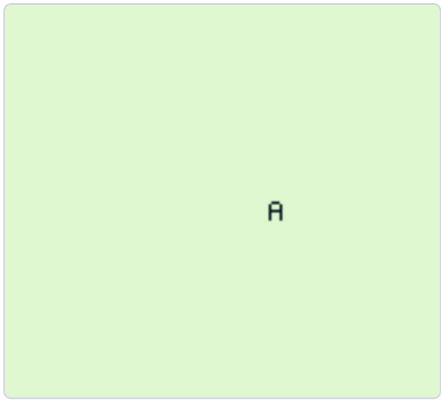
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

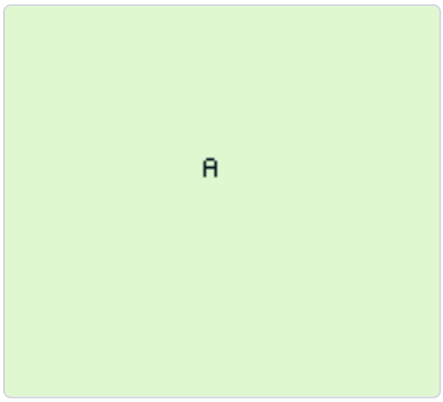
la toute première image — la console vient de démarrer — image 1



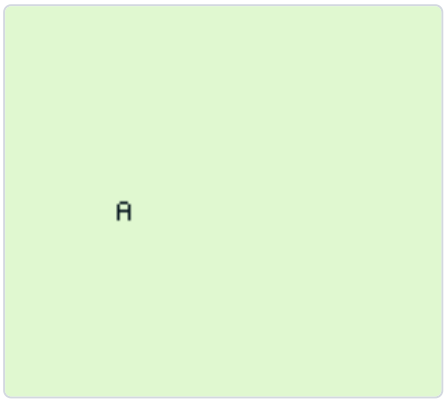
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Huit tours autour du milieu de l'écran, de plus en plus grands, jusqu'à un carré qui touche presque les bords.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les coins grandissent : (12, 10), (14, 12), (18, 16)
- ✓ le plus grand va jusqu'en haut à gauche (2, 0)
- ✓ jamais hors de l'écran : ni colonne 19, ni ligne 17
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.51.1. Jusqu'au plus grand : une boucle — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++